

FICHE DE TD 3/3 • DIFFICILE

Informatique • 1re TI

Programmation procédurale en langage C

Module 2 : Programmation

Consigne : Les exercices suivants portent sur l'analyse, la conception et la mise en œuvre de programmes C complets. Toute réponse doit être justifiée par un raisonnement rigoureux ou un tracé d'exécution. L'utilisation de `math.h` est autorisée.

EXERCICE 1 Analyse numérique et compilation : recherche de racine par dichotomie

On souhaite programmer la méthode de dichotomie pour approcher une racine de la fonction $f(x) = x^3 - 2x - 5$ sur l'intervalle $[2, 3]$ avec une précision $\varepsilon = 10^{-6}$.

1. **Compilation et types :** Expliquez la différence fondamentale entre un langage compilé (comme C) et un langage interprété lors de la détection d'une erreur de type (ex : affectation d'un `double` dans un `int`). Quel rôle joue l'éditeur de liens (linker) vis-à-vis de la fonction `sqrt` de `math.h` ?
2. **Algorithme et traduction :** Écrivez l'algorithme complet de la dichotomie (variables, initialisation, condition d'arrêt, mise à jour des bornes). Traduisez-le en C strictement standard (C99) dans une fonction `double dichotomie(double a, double b, double eps)`. Gérez le cas où $f(a) \times f(b) > 0$.
3. **Analyse d'erreur et types :** Dans votre code, justifiez le choix du type `double` plutôt que `float` pour les bornes et le milieu. Quelle est la valeur maximale théorique du nombre d'itérations nécessaires pour atteindre ε ? Vérifiez-la par un tracé manuel des 3 premières itérations (tableau : $a, b, m, f(m)$).
4. **Exécution pas à pas et optimisation :** Proposez une réécriture de la boucle `while` en boucle `for` équivalente. Discutez de l'impact de l'instruction `register` (obsolète mais conceptuelle) sur la variable `m` dans un contexte d'optimisation compilateur (-O2 vs -O0).

EXERCICE 2 Automate à états finis : Analyseur de trames de capteur industriel

Un capteur envoie des trames ASCII sur port série au format : `<ID>:<VALEUR>;<CHECKSUM>\n`. Exemple : `T1:23.5;78\n`. L'ID est un caractère majuscule suivi d'un chiffre. La valeur est un réel signé. Le checksum est la somme modulo 256 des codes ASCII des caractères précédant le `';`.

1. **Conception :** Modélisez l'analyseur par un diagramme d'états (texte ou tableau) avec 5 états minimum : `ATTENTE_ID`, `LECTURE_ID`, `LECTURE_VALEUR`, `LECTURE_CHECKSUM`, `VALIDATION`. Définissez les transitions et les actions (accumulation checksum, construction valeur).

2. Implémentation C : Écrivez le programme principal `main()` qui lit caractère par caractère (`getchar()`) et implémente cet automate via un `switch` imbriqué dans une boucle `while`. Utilisez des variables `static` ou globales pour l'état courant et les accumulateurs. Affichez `TRACE VALIDE : ID=..., VAL=... ou ERREUR CHECKSUM`.
3. Robustesse et `scanf` : Pourquoi l'utilisation de `scanf("%c:%f;%d; ...)` est-elle dangereuse et inadaptée ici (buffer, blocage, synchronisation)? Comment votre automate gère-t-il une trame tronquée ou un caractère inattendu (ex : lettre dans la valeur)?
4. Extension : Modifier l'automate pour accepter plusieurs trames successives sans redémarrage. Quelle variable doit être réinitialisée et où exactement dans le code (quel état)?

EXERCICE 3 Synthèse : Simulateur de système de commande discret

On modélise un système du premier ordre : $y_{k+1} = a \cdot y_k + b \cdot u_k$ avec $a = 0.9$, $b = 0.1$, $y_0 = 0$. L'entrée u_k est une consigne carrée : $u_k = 10$ si $k < 50$, sinon $u_k = 0$. On simule $N = 100$ pas.

1. Structure modulaire : Sans écrire de fonctions utilisateur (hors `main`), structurez le code en blocs logiques séparés par des commentaires : Déclarations/Constantes, Initialisation, Boucle de simulation, Sortie/Statistiques. Justifiez l'usage de `const` pour a , b , N et `#define` pour les seuils.
2. Boucles et conditions : Implémentez la simulation avec une unique boucle `for`. À l'intérieur, utilisez un `if/else` pour u_k . Calculez et affichez (format `%03d %.4f %.4f`) : l'indice k , u_k , y_k .
3. Analyse critique : La consigne change à l'itération 50. Proposez une réécriture utilisant deux boucles `for` successives (sans `if` interne). Comparez le nombre d'opérations arithmétiques et de sauts conditionnels des deux versions. Laquelle est préférable en temps réel dur?
4. Précision et dérivée : Ajoutez le calcul de la dérivée discrète $\Delta y_k = y_k - y_{k-1}$ (avec $\Delta y_0 = 0$). Détectez et affichez l'indice k où $|\Delta y_k|$ est maximal. Expliquez pourquoi ce maximum ne se produit pas exactement à $k = 50$ mais légèrement après, en lien avec la constante de temps du système.

EXERCICE 4 Étude comparative : Tris élémentaires et complexité

On dispose de 3 algorithmes de tri pour un tableau de n entiers (tableau passé en paramètre, taille $n \leq 100$) : Tri à bulles, Tri par sélection, Tri par insertion.

1. Code C : Écrivez les trois fonctions `void tri_bulles(int t[], int n)`, `void tri_selection(int t[], int n)`, `void tri_insertion(int t[], int n)`. Utilisez

uniquement les boucles `for` et `while`, indices entiers, et une variable temporaire `tmp` pour l'échange.

2. **Comptage d'opérations** : Pour chaque tri, déterminez la complexité en temps (notation $O(\cdot)$) au pire cas et au meilleur cas. Pour $n = 10$, calculez le nombre exact de comparaisons `if (t[j] > t[j+1])` (ou équivalent) effectuées par le tri à bulles non optimisé (pas de drapeau `swapped`) sur un tableau déjà trié.
3. **Expérimentation** : Écrivez un `main()` qui : (1) remplit un tableau de 20 entiers aléatoires (`rand() % 100`), (2) en fait 3 copies, (3) lance les 3 tris sur chaque copie, (4) vérifie l'ordre croissant final par une fonction `int est_trie(int t[], int n)` à écrire.
4. **Analyse critique** : Pourquoi le tri par insertion est-il souvent plus rapide que le tri par sélection sur des tableaux quasi triés, bien qu'ils aient la même complexité quadratique au pire cas ? Reliez votre réponse au nombre d'écritures en mémoire (affectations) et au comportement du `while` interne.